

1) User Uploads Tender

API: `POST /api/tender/upload`

User uploads:

- Tender PDF
- Optional supporting images / annexures

System generates:

- `tender_id`
- file checksum (SHA256)
- upload timestamp

File saved locally:

`storage/tenders/{tender_id}/original.pdf`

MongoDB save:

```
{
  "_id": "tender_id",
  "name": "Tender Name",
  "file_path": "storage/tenders/{tender_id}/original.pdf",
  "checksum": "...",
  "status": "UPLOADED"
}
```

Response immediately:

```
{
  "tender_id": "...",
  "status": "PROCESSING"
}
```

Async pipeline starts.

2) Parse Document

System reads PDF.

Then extract content using different parsers:

Table Parser [take from [chunking.py](#) and update service file accordingly]

Paragraph Parser [take from [paragraph_chunking.py](#) and update service file accordingly]

List Parser [take from [list_chunking.py](#) and update service file accordingly]

OCR Parser [take from [ocr_chunking.py](#) and update service file accordingly]

3) Create Unified Evidence Pool [Already created in `evidenve_pipeline.py` just need to use it in service keep it in service file]

Merge all parser outputs into one normalized evidence pool.

Each record:

```
{
  "evidence_id": "...",
  "source": "table / paragraph / list / ocr",
  "kind": "...",
  "page": 1,
  "heading": "...",
  "text_raw": "...",
  "text_norm": "...",
  "bbox": [...],
  "confidence": 0.92
}
```

Save locally:

storage/tenders/{tender_id}/evidence_pool.json

Save Mongo: One evidence = one document. Decide schema according to the generated evidence and it will be same throughout the process

tender_evidence

Status:

PARSED

4) Canonical Criteria Generation [already happening in evaluation_pipeline.py]

Save locally:

storage/tenders/{tender_id}/canonical.json

Status:

CANONICAL_READY

5) LLM Call:

In the canonical file we have various categories so make a llm api call category wise and store the result from the llm into mongo db complete document for category on tender id.

Prompt:

You are a **Tender Criteria Normalization Engine**.

Your task is to clean, normalize, deduplicate, and map extracted tender criteria into canonical structured criteria.

Context

I have already parsed a tender PDF into structured JSON.
The extracted data may contain:

- duplicate entries
- OCR noise
- Hindi + English mixed text
- broken headers
- repeated paragraph/table/list evidence
- irrelevant lines
- partially extracted numeric values
- incorrectly mapped entries

Your job is to convert noisy extracted records into **clean canonical evaluation criteria**.

Rules

1. Remove garbage / OCR noise

Ignore:

- corrupted unicode
- incomplete text
- random fragments
- broken Hindi text
- meaningless repeated lines

2. Merge duplicates

If multiple records represent same criterion:
merge them into one canonical criterion.

Example:

"Minimum Average Annual Turnover of bidder"

"Average Annual Financial Turnover"

"Bidder Turnover"

→ one canonical criterion:

"Bidder Annual Turnover"

3. Normalize naming

Convert raw header into clean canonical name.

Examples:

Past Performance → Past Supply Performance
OEM Turnover → OEM Annual Turnover
GST Registration → GST Compliance

4. **Extract structured values**

Identify:

- threshold_value
- unit
- operator
- years
- percentage
- required_documents
- exemptions
- mandatory(true/false)

Examples:

1 Lakh → 100000 INR

50 % → 50 percent

3 Years → 3 years

5. **Identify evaluation type**

Choose one:

- numeric_threshold
 - boolean_requirement
 - document_requirement
 - experience_requirement
 - compliance_requirement
 - technical_requirement
 - weighted_requirement
-

6. **Generate matching aliases**

Create aliases bidder submissions may use.

Example:

Bidder Annual Turnover aliases:

- turnover
- annual turnover

- financial turnover
- audited turnover
- bidder turnover

7. **Confidence score**

Return confidence:

0–1

Output format (STRICT JSON)

```
[
  {
    "canonical_name": "",
    "category": "",
    "criterion_type": "",
    "operator": "",
    "expected_value": null,
    "unit": "",
    "period_years": null,
    "mandatory": true,
    "required_documents": [],
    "aliases": [],
    "supporting_evidence_ids": [],
    "confidence": 0.95
  }
]
```

Only output JSON.

Now process this category:

<PASTE_CATEGORY_JSON_HERE>

5) Create Search Index

Generate:

TF-IDF vectors []

For lexical matching. [only for canonical]

Semantic embeddings [for both canonical and evidence]

For semantic matching + chatbot retrieval.

Create embeddings for:

- canonical criteria
- tender evidence

Save Mongo vector collection.

Status:

INDEXED

6) Tender Knowledge Base Ready

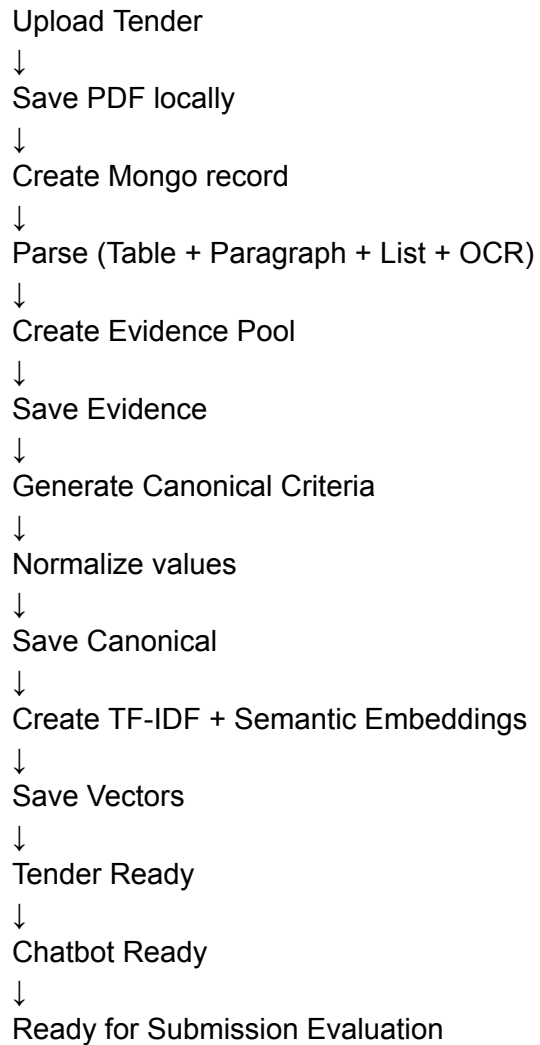
System now has:

- ✓ Original tender file
- ✓ Evidence pool
- ✓ Canonical criteria
- ✓ TF-IDF index
- ✓ Semantic embeddings
- ✓ Audit references

Tender is now:

READY_FOR_SUBMISSIONS

Final Flow



Principle: Every extracted criterion must remain traceable back to original evidence for explainability and auditability.